storygit

Version control for story state

Arush Gumber

Abstract

storygit is an AI-assisted storytelling system that develops a one-line premise into a structured serial — Story, Episodes, Scenes, Beats, Prose — while keeping the writer in authority over every change. Its claim is that the hard part of long-form AI-assisted fiction is not generation but *state*: what is true, who knows it, what depends on it, and what a given edit therefore invalidates. storygit models the story as a versioned typed state graph, makes every AI action a reviewable diff against it, propagates edits by *marking* affected nodes rather than rewriting them, checks continuity graph-first before it ever asks a model, and learns the writer's taste from their accept, reject, and edit signals. This document states the problem, the design, the decisions and their rejected alternatives, and the measurements.

1 The problem

A writer starts with a sentence: *a powerless orphan discovers an ability that could change the balance of power in a world at war.* They want to end with a serial — tens or hundreds of episodes, coherent, in their voice, with the mechanics that make a listener press *next episode*. The naive form of AI assistance, prompt to story, fails at this in five reliable ways.

State drift. Facts established early are forgotten or contradicted later. The model has no ledger of what it has committed to, so episode 14 puts a character in a city they left in episode 9. The sharpest version of this failure is *epistemic*: a character acts on information the story has never given them. A reader forgives an inconsistent hair colour; they do not forgive a character who somehow already knows the secret.

Broken edit semantics. The writer changes one scene. Either nothing downstream responds — the story now contains a contradiction nobody flagged — or the system regenerates the plan and destroys work the writer had already approved. Both come from the same absence: nothing records what depends on what. Fabula's own study [10] reports exactly this, from several writers independently.

Agency erosion. When the system proposes whole drafts, the writer's role collapses into judging them. Fabula's participants described becoming "an arbiter of good or bad" rather than an author [10]. That is a worse job than writing, and it is the reason a tool can be impressive in a demo and unused in a month.

Exploration overwhelm. Offering more alternatives is not the same as offering more choice. Unlabeled variants cost more to read than they save, and a writer will stop opening them.

No learning. The thirtieth iteration repeats the first iteration's mistakes. Nothing in the system is a model of *this* writer's taste, so every correction has to be made again.

To these, serialized audio fiction adds a sixth: **no serial mechanics**. Hooks, cliffhangers, recaps, and the open threads a listener is waiting to see paid off are the retention surface of the product, and a general-purpose story generator does not track any of them.

2 Thesis

Treat the story as a **versioned, typed state graph**. Make every AI action a **reviewable diff** against that graph rather than a block of replacement text. Make dependencies **explicit**, so that an edit propagates by *marking* what it affected and never by rewriting it. Let the writer **own the objective**, in their own words, rather than ranking against the system's. And **learn the writer's taste** from the signals they are already producing — what they accept, what they reject, and how they edit.

The division of labour follows: **the AI handles coherence and mechanics; the human handles taste, voice, and retention.**

Stated as a difference: Fabula is a chain of prompts with a user interface. storygit is a state machine with a learning loop. The chain is fast to build and pleasant to demonstrate; the state machine is what survives episode 40.

3 Fabula, and the gaps this exploits

Fabula [10] (DeepMind, 2026) builds a two-level hierarchical plan — a story plan of scenes, then beat plans per scene — and then a script, using prompt-chaining orchestrators with structured output. It is editable at every level, with top-down propagation and partial bottom-up updates. It ships 26 orchestrator variants, an auto-rater over 63 narratology guidelines that argues before it rates (to reduce position bias) and correlates 0.83 with human preference after tuning, a generate–critique–select search loop, and suggestion diversity implemented as embedding cosine against past suggestions. It was evaluated with 42 writers — 18 design interviews and 25 three-hour sessions — plus hundreds of testers. It is the closest published system to this problem, and the immediate predecessor of these questions; Dramatron [9] and Wordcraft [14] are the hierarchical-generation and writer-in-the-loop lines it builds on, and the creativity-support literature [4] supplies the vocabulary for judging such tools.

The study's findings matter more than its architecture, and they are unusually specific. Writers liked the structure, the scene breaking, the character questions, and the plan-beside-script view for restructuring. They disliked generic prose and poor handling of style, irony, and satire. They found the system *rigid*: they could not add a character locally, edits rewrote the whole plan, stale downstream scenes were not flagged. They asked for locks, insert and delete controls, progressive build-up, and an "absurdity dial". And the authors' own conclusion is that the system is most useful as feedback on the writer's own script, not as a generator.

Three of those complaints — edits rewriting the plan, stale scenes going unflagged, and the inability to add a character locally — are the same missing abstraction seen from three angles: there is no explicit dependency graph. storygit builds that abstraction first and derives the rest from it.

Gap	What storygit does about it
No dependency tracking; edits rewrite the plan	Beats declare produces / consumes; propagation *marks* affected nodes and never rewrites them
No per-character knowledge state	Epistemic edges knows(character, fact, since beat), checked before a character may act on something
No learning from writer signals	A preference layer trained on accept / reject / edit, with the writer's own criteria as features
No serial mechanics	Episode hooks, cliffhangers, recaps, and an open-thread ledger with last-touched dates
Diversity is "do not repeat the last one"	Axis-conditioned sampling with MMR or DPP selection over a quality-weighted similarity kernel

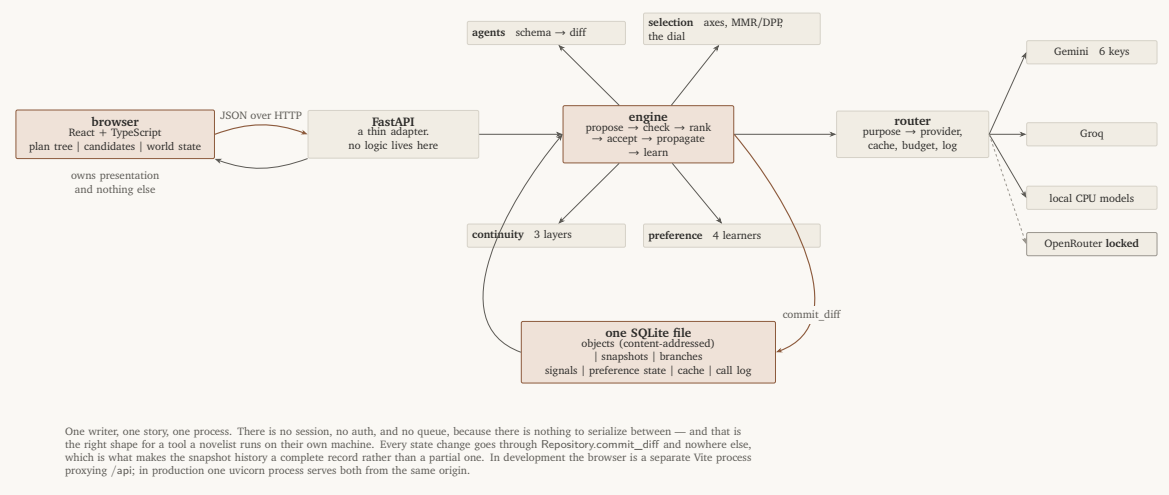

Figure 1: One writer, one story, one process. There is no session, no auth, and no queue, because there is nothing to serialize between. Every state change goes through `Repository.commit_diff` and nowhere else, which is what makes the snapshot history a complete record rather than a partial one.

4 Architecture

Five components and an interface. Everything below the interface is a pure Python library with no web machinery in it — which is why most of it is testable with no network and measurable with no server.

Package	What it does
`domain/`	The typed state and the 31 diff operations that are the only way to change any of it. Frozen models, a pure `apply`, typed errors.
`store/`	Content-addressed objects, snapshot manifests, branch pointers, three-way merge. Git's data model, in one SQLite file.
`graph/`	Declared dependency edges, propagation that marks and never rewrites, and the entity-scoped slices generation prompts consume.
`providers/`	One interface over four backends, with key rotation, a read-through cache, a budget guard, and a call log. No key is ever logged.
`agents/`	Schema-constrained generation converted into typed diffs, plus fact extraction from accepted prose.
`selection/`	Named axes, MMR / DPP / top-k behind one signature, and the dial.
`continuity/`	The three checker layers, the bible diff, and the periodic audit.
`preference/`	Exemplars, edit mining, the voice model, the ranking head, the bandit.
`eval/`	Simulated writers, injected ground truth, metrics, ablations, the gallery recorder.
`api/` + `frontend/`	A thin HTTP adapter and a five-tab interface. No logic lives in either.

4.1 The interface, and why it is part of the argument

The interface is where the design either lands or does not. Fabula's finding that writers became "an arbiter of good or bad" is not fixed by a better model; it is fixed by changing what the writer is asked to do.

Diffs, not drafts. A writer reading a wall of replacement text is judging prose. A writer reading *"introduces Mara; changes Kael's goal; would mark 2 downstream beats stale"* is authorizing a change. Same underlying output, entirely different job.

Named directions, not variants. Three options labelled *raise the stakes / slow down / introduce a complication* cost one decision. Three unlabelled paragraphs cost three readings, and a writer who has done that twice stops opening the third.

The blast radius before the decision. Every candidate says how many downstream nodes it would mark. That one line is the difference between a system a writer trusts with a manuscript and one they do not, because it makes the consequence visible while it is still avoidable.

Learning that is visible. Every rule mined from the writer's edits lands in the ledger, is shown, and can be deleted, and enters prompts only after being observed twice. A system that quietly adjusted its own prompts from inferred preferences would work slightly better and would be untrustworthy the first time it inferred wrong — and the writer would have no way to find out.

Nothing the system can do is API-only. A capability the writer cannot reach is a capability the product does not have, however well it is tested. The whole-graph audit, the three-way merge with its conflicts, the snapshot chain, striking a fact, merging a duplicate entity, deleting a mined rule — each is a control in the tool, not just a route. This is checked rather than asserted: a test fails if any method in the API client is called by no component, and another fails if any state operation is constructed by nothing outside the two modules that define and apply it.

The five tabs run in the order the story is told: problem, architecture, tool, proof, numbers.

5 The state model

Five things are stored, and everything else is derived from them.

The plan tree. Story, Episode, Scene, Beat, Prose. Every node carries the four planning questions Fabula found writers actually use — what happens, what the audience learns, how they feel, where and when — plus a review status (`draft`, `accepted`, `locked`, `stale`) and, when stale, the reason. Episode nodes add the serial mechanics: hook, cliffhanger, recap of the previous episode, threads carried in and left out, writer-set goals, and a target length. Beat nodes add `produces` and `consumes` — the declared fact dependencies that make propagation exact.

The world graph. Entities (characters, places, objects, factions) with an alias table, and facts as edges between them. A fact carries a validity interval over the global beat order, so a fact that changes does not overwrite its own history: Kael is in Ashfall for beats 3 through 7 and elsewhere afterwards, and both are retrievable at their own times. Predicates come from a closed vocabulary (`location`, `alive`, `injury`, `possesses`, `relationship`, `relationship_status`, `goal`, `secret`, `trait`, `faction`) plus a free-text `note`. Closed is what makes the first layer of continuity checking a dictionary lookup rather than a model call.

Using an explicit world-state graph to keep generated narrative consistent is an established idea [1]; what is added here is temporal validity, the epistemic layer, and the declared dependency edges that make edits propagate.

Epistemic edges. `knows(character, fact, since beat)`, tracked separately from the facts themselves, because the difference between what is true and what a character has been told is the single richest

source of continuity errors in generated fiction.

The open-thread ledger. Each thread records when it was opened and last advanced, so "you opened the sister subplot in episode 2 and have not touched it in nine episodes" is a query, not a judgement.

The writer ledger. Locks, hard constraints, style notes, writer-defined scoring criteria in the writer's own words, rejected directions, and the coherent-to-surprising dial. This is the object that keeps the human in authority, and everything in it is visible and deletable — including the rules the system mined from the writer's own edits, which must never influence generation invisibly.

Provenance is recorded per sentence (`human`, `ai`, `ai_edited_by_human`), because the interesting case is mixed: the AI proposed a paragraph and the writer rewrote two sentences of it. A node-level flag would lose exactly the information the writer cares about, and the authorship ratio shown in the interface would be a lie.

5.1 Snapshots, branches, and why the project is called storygit

Every object is stored once under the SHA-256 of its canonical JSON. A snapshot is a *manifest*: a mapping from logical id to content hash, plus a parent pointer, the author, and the intent line. A branch is a name pointing at a snapshot.

Three properties fall out for free. Versions are cheap: committing a change that touches three nodes writes three object rows and one manifest, so a five-hundred-snapshot history of a two-hundred-node story costs the changed nodes, not five hundred copies. Structural diff needs no heuristics: what changed between two versions is a set comparison over two manifests. And the whole thing is reproducible: identical content yields an identical manifest, so "apply the same diff twice and get the same hashes" is a test rather than a hope.

Merging is three-way at object granularity. Where only one side changed an object, that side wins. Where both changed it differently, the merge returns a conflict. It never guesses which version of a scene the author meant.

6 The proposal engine

Every AI action is a **diff**: a list of typed operations against the current state, carrying a rationale and a delta summary in English. The writer reads *"introduces Mara; changes Kael's goal; would mark 2 downstream beats stale"* and decides. They never read a wall of replacement text and guess what it changed.

This is the direct answer to two of Fabula's findings at once. "I could not add a character locally" is, in this model, a single `AddEntity` operation. And "editing one scene rewrote the whole plan" cannot happen, because a diff that would rewrite the plan is visibly a diff that rewrites the plan.

6.1 Progressive levels

Generation is a function `propose(level, state slice, intent, k)`. The levels run seed → premise → episodes → scenes → beats → prose, and the writer settles each before the next exists. This is Fabula's progressive-build-up lesson taken literally: writers rejected systems that produced a whole plan before they had agreed to its premise.

Every level has a schema the model must fill, and the schema is where the requirements live. A beat proposal cannot come back without saying which facts it establishes and which existing facts it relies on, because those are required fields — so the dependency graph that propagation walks

cannot quietly go unmaintained. The same schema requires, for each new fact, the list of characters who *learn* it at this beat: not everyone present, and not everyone it is true of.

Every field carries a length bound, forwarded into the provider's own response schema. This keeps candidates readable, and it closes a failure mode observed in the first live test: a small model fell into a repetition loop and filled a field until it hit the token cap, turning a good candidate into truncated JSON.

6.2 What the model is allowed to see

Never the story. Generation is fed an **entity-scoped slice**: the entities in scope, the facts about them *valid at that beat* with their ids, who knows what, the open threads, the plan path down to the insertion point, hard constraints from locks, the writer's style notes, and the writer's own criteria. A few hundred tokens instead of tens of thousands.

The important difference from ordinary retrieval-augmented generation is not the size, it is the kind of answer. Chunk-and-embed retrieval returns text that is *approximately relevant*; a query against a typed graph returns what is *exactly true, at this point in the story, about these entities*. That precision is what makes the continuity claims in §8 checkable rather than aspirational.

6.3 Extraction, and the closed vocabulary

When prose is accepted — whether the model wrote it or the writer typed it — a cheap model reads it back and returns structured facts. A hand-written beat therefore participates in continuity exactly like a generated one: the system does not generate anything, it only reads what was written and updates the graph.

Predicates come from a closed vocabulary of ten, plus a free-text note. Closed is what makes the first checker layer a dictionary lookup and an equality test rather than a semantic judgement, and the escape hatch isolates exactly the residue that needs the more expensive check.

Names are canonicalized conservatively: exact match after normalization, otherwise a new entity *and* a note telling the writer what it might duplicate. No similarity threshold is used. A fuzzy match that is occasionally wrong would weld two characters together silently, which is the failure this whole system exists to prevent.

One routing table maps purpose tags to backends: prose and judging to Gemini, extraction and classification to Groq's small fast model, embeddings and NLI to local CPU models, and a metered provider that is locked. Around the chosen backend sit a read-through cache, a budget guard, and a call log that records tokens for every call.

Three details are load-bearing. Gemini's six free-tier keys carry per-`(key, model)` cooldowns, because free-tier quota is per model per key — exhausting a key on one model says nothing about its allowance on another. The cache key includes a per-sample index, so the six axis-conditioned candidates of §**??** are six entries rather than one. And non-prose purposes fall back across providers when one is down, while prose does not: a beat silently served by an 8B model is a quality regression the writer cannot see the cause of.

6.4 The loop

1. The writer states an intent at a node.
2. The engine slices the state, samples k candidates, converts each into a typed diff, and previews what each would invalidate.

3. The writer accepts, edits, or rejects. Editing keeps the writer's words and records the before/after pair; rejecting records the direction so it is not proposed again.
4. Accepting commits the diff, extracts facts from any new prose, and commits the propagation marks as a second snapshot.
5. Every action is recorded as a signal, along with the alternatives it was shown beside — which is what makes an accept a *preference* rather than an event.

7 Propagation

Dependency edges are declared, not inferred: a beat states the facts it establishes and the facts it relies on, and an edge from beat *A* to beat *B* exists exactly when *B* consumes something *A* produced. When an accepted change alters or removes a fact, the system walks the consumers transitively and *marks* them. It never rewrites them.

Three rules make this safe to leave running.

1. **Locked nodes are never marked**, and the walk does not pass through them. A locked beat is not going to change, so nothing it produces can change either. The operational meaning of a lock is broader than that: every fact a locked node establishes is exported into every subsequent prompt as a hard constraint.
2. **Human prose is flagged for review, not staled.** The system does not tell an author that their own sentences are out of date; it tells them something they relied on moved, and leaves the judgement where it belongs.
3. **Nothing is ever rewritten.** The writer chooses: regenerate (previewed as a diff, respecting locks and current facts), edit by hand, or dismiss — "it still works". Regenerating a whole stale subtree exists, as an explicit, confirmed, previewed action.

Because the marks are emitted as ordinary status operations, staleness flows through the same snapshot machinery as every other change. There is no side channel, and the history shows why each node was marked and by which upstream fact.

The same walk, run against a candidate that has not been accepted, is what produces the line the writer reads under every proposal: *"would mark 2 downstream beats stale"*. The blast radius is visible before the decision, not after it.

8 Continuity checking

Three layers, cheapest and most certain first. Each check is pushed to the cheapest layer that can decide it, and each layer's recall is reported separately — which is what turns "it keeps the story consistent" from an assertion into a measurement.

Layer 1 — deterministic. For each new fact, look up existing facts with the same subject and predicate, valid at that beat. With a closed vocabulary that is a dictionary lookup returning zero to three facts, and the contradiction check is equality. It catches conflicting single-valued facts (a character in two places, alive and dead, in two factions), dead characters who act, two holders of one object, and — the important one — **epistemic violations**: a beat that relies on a fact, performed by a character who has not been told it. The flag says whether they learn it later ("learns it in *Beat D*") or never.

It deliberately does not flag a properly ended fact being replaced (a change, not a contradiction), a fact re-established with the same value (repetition), two goals at once (a character), or a character acting on their own secret.

Layer 2 — local NLI. The residue is the free-text note predicate and multi-valued facts whose object strings differ. A DeBERTa cross-encoder [6] fine-tuned on MNLI scores each pair for contradiction — the same instrument, and for the same reason, that the summarization-consistency literature uses to detect claims a source does not support [8]. A cross-encoder rather than a bi-encoder because the distinction we need is exactly the one a bi-encoder structurally cannot make: "Kael is in Ashfall" and "Kael is not in Ashfall" embed almost identically, being about the same thing, but a model that reads both sentences together sees the negation.

The threshold is calibrated, not chosen. On the local checkpoint a blunt contradiction scores 1.00, a paraphrase 0.00, and a real but universal-versus-specific contradiction 0.43 — so a 0.5 threshold would miss the third. The default is 0.35, and §11 sweeps it.

Layer 3 — soft judge. Motivation plausibility and tone, plus the ranking score. It argues before it rates: one sentence of reasoning is required before a number, which is Fabula's own finding about their auto-rater and matches the general result that a score conditioned on an explanation is better calibrated. It scores the writer's own criteria, in the writer's words, alongside fixed narratology axes.

8.1 What a flag must carry

Two properties are non-negotiable. **Every flag cites its establishing beat** — "Kael cannot be in Ashfall here" is useless; "Kael was established in Kell in *The Warden's Offer*" is actionable. And **nothing is ever auto-fixed**. A contradiction can be exactly what the writer meant: characters lie, narrators are unreliable, a flashback contradicts the present on purpose. A system that silently repairs those is worse than one that says nothing.

Hard flags (deterministic) and soft flags (opinions) are structurally distinct in the data model, sorted apart, and rendered differently.

8.2 The bible diff

On accept the writer sees what changed in the world, in sentences:

```
+ Kael keeps a secret:  he can hear the ash.
~ Kael is at Ashfall.  (no longer true from here)
- Kael has the Warden's token.
```

They can strike any line. Striking is not an undo: it produces a normal diff — delete if the fact was never true, end it if it stopped being true — which propagates like any other change and re-runs layer 1 on whatever depended on it. Even correcting the machine goes through the same reviewable path as everything else.

8.3 The audit

Per-accept checking is incremental and can miss slow drift: a contradiction assembled over ten episodes where no single accept was wrong. A periodic audit walks the whole graph through layers 1 and 2. It also answers a question no per-fact check can — which threads are being dropped. A thread opened in episode 2 and untouched since episode 3 is not a contradiction, so nothing else in the system would ever mention it, and for a serial it is the most expensive kind of mistake.

9 Diversity, and the dial

Sampling three continuations from one prompt gives three phrasings of one idea, and the writer has no choice to make. Fabula's answer was to penalize similarity against previous suggestions, which produces variants that differ but are not *nameably* different — and their own finding is that unlabeled alternatives cost more to review than they save. Diversity in the embedding space is not diversity in the interface.

Axes. Seven named conditioning instructions — raise the stakes, slow down, subvert the expectation, shift the point of view, deepen the interiority, introduce a complication, pay off an open thread. Six are rotated deterministically per node, sampled in parallel, and the label travels with the candidate to the screen.

Selection. MMR by default: $\lambda q_i - (1-\lambda)\max_{j\in S}\mathrm{sim}(i,j)$, so the second pick is the best candidate that is not the first one again. A determinantal point process is implemented as a drop-in alternative: with kernel $L = \mathrm{diag}(q)\,S\,\mathrm{diag}(q)$, the determinant of a subset is the squared volume its quality-scaled vectors span — large when they are good *and* mutually dissimilar, zero when two are parallel. The diversity trade-off is the structure of the distribution rather than a tuned parameter [7, 3]. Plain top-k is the ablation baseline. All three share one signature, so the comparison is one config value.

The dial. Fabula's writers asked for an "absurdity dial". The request is deeper than it sounds: it asks to control the objective, not the output. A writer at 3am on episode 40 wants safe continuations; the same writer opening a new arc wants to be surprised. Take the greedy temperature-0 continuation as "what this model would obviously do next"; each candidate's surprise is its embedding distance from that; effective quality is $(1-d)\tilde{q} + d\tilde{s}$. At $d = 1$ the system is explicitly selecting against what the model would have done anyway. One extra cached call per node.

Candidates arrive already checked. Layer 1 runs on each candidate's would-be facts — the diff is applied to a scratch state and checked there — so a candidate that contradicts the bible reaches the writer with the contradiction attached. Hard flags push a candidate down the ranking but never remove it.

10 The preference layer

Iteration thirty should not repeat iteration one's mistakes. Four learners over the same signal — accept, reject, edit — feed one ranking.

10.1 The signal is a comparison

Every accept is recorded with the alternatives that were on screen. "The writer liked A" is weak; "the writer preferred A to B and C, having seen all three" is a comparison, and comparisons are what Bradley-Terry is defined on. Pairs are formed only *within* one shown set — comparing a candidate accepted on Monday against one rejected in a different scene on Friday would compare two different questions. An edited-then-accepted win is weaker evidence than a clean accept and is weighted accordingly.

10.2 Four learners, four data appetites

A writer produces about twenty comparisons in a session, which is nowhere near enough to fit one model that does everything. So each signal is exploited by the cheapest mechanism that can use it, and each contributes as soon as it has enough.

Learner	Needs	What it contributes
Exemplar retrieval	1 accept	The writer's own sentences in the prompt, and their rejected ones as negatives
Edit direction	1 edit	Mean embedding shift from the model's version to the writer's; score by projection onto it
Edit mining	2 similar edits	Plain-language style rules, into the visible ledger
Bradley-Terry head	∼10 comparisons	The ranking
Voice model	∼dozens of texts	"Sounds like this writer"
Thompson bandit	∼20 shown sets	How much of the shown set to spend on exploration

10.3 The head

$P(a \succ b) = \sigma\big(w^\top (f_a - f_b)\big)$ — logistic regression on feature differences, the intercept cancelling. Convex, milliseconds in NumPy, and the fitted w is directly readable, which is what lets the interface say *"you have been choosing for specificity and against length"*. It is the same construction as an RLHF reward model [2, 5, 11] minus the policy-gradient half; at one writer and a few dozen decisions, ordering candidates is the right ambition.

The feature vector is ten numbers: four judge sub-scores on fixed narratology axes, the writer's own criteria, continuity, voice cosine, edit-direction projection, length, and dialogue ratio. Few and interpretable on purpose — a high-dimensional head would memorize twenty comparisons rather than learn from them. They are also the features the simulated writers' hidden weights are defined over, which is what makes §11's central question answerable.

10.4 Voice

A judge can score whether a passage is well constructed; it cannot say whether it sounds like *this* writer, because that is an identity rather than a quality. A frozen bge-small encoder plus a 384×128 projection trained with InfoNCE [13]: anchors are prose the writer wrote or edited, positives are what they accepted, negatives are what they rejected and the pre-edit versions of what they changed. Frozen encoder because a session's worth of prose would damage an encoder far more than it would personalize it; the projection only has to find which directions in an already-good space this writer cares about.

10.5 Exploration

A ranker that always shows its top three converges: the writer only ever sees what the model already believes, so the model never learns it was wrong. Two arms over the composition of the shown set — three safe, or two safe plus one high-surprise — chosen by Thompson sampling [12]. Exploration anneals itself out of the posterior width, with no schedule to tune; ε-greedy would keep paying a fixed exploration cost forever and would explore uniformly, spending as much on an arm it is sure about as on a marginal one.

10.6 The data regime, stated plainly

The head starts from a prior fitted across synthetic proto-personas and adapts per writer from there, because twenty comparisons cannot fit a model from scratch and there is no population of real writers to fit a prior on.

That prior cannot contain real human taste. Nothing in its data ever met a reader. What it can contain is the structure of the problem — that features have consistent signs, that the signal is noisy, that some axes matter more than others. The bet is that the adaptation *machinery* transfers even though the values do not, and the bet is measured rather than asserted: a prior-initialized head reaches 0.775 held-out pairwise accuracy after a fresh writer's first ten decisions, against 0.645 from uniform initialization. Were that lift zero, the prior would be worthless and should be dropped.

There is a deeper circularity worth naming: the preference layer is validated against simulated writers made of the same feature vector the head is fitted on. That is why the evaluation measures *recoverability* — whether the fitted weights correlate with the hidden ones — rather than claiming the system learns taste. Recoverability is a real property of the machinery. Taste is not something this evaluation can measure at all; only a human study would settle it.

11 Evaluation

Every claim in this document is either measured or explicitly marked as unmeasured. This section is the first kind; §14 is the second.

11.1 Simulated writers

Four personas, each **a hidden weight vector over exactly the feature space the preference head is fitted on**, plus style quirks and forbidden moves — absolutes a linear preference cannot express, and which real writers have.

Persona	Weighted towards	Character
the Serialist	momentum, consequence	Writes for retention; no flashbacks, they stall
the Minimalist	specificity, *negative* on length	Short, concrete, exposition-averse
the Maximalist	voice, interiority, length	Dial at 0.70; dialogue low
the Controller	continuity, own criteria	Accepts rarely, edits heavily, locks constantly

Defining the personas over the head's own feature space is what makes the central question measurable: not *did acceptance rise* — a degenerate system achieves that by proposing the same safe thing repeatedly — but *did the fitted weight vector correlate with the hidden one*. Three constraints keep it from being circular, the first asserted in a test: the engine is never told the hidden weights; the persona sees only what the engine chose to show; and the model that performs the persona's edits is told their *style*, never their objective.

11.2 Ground truth by construction

Recall needs a denominator. One contradiction of each class is injected into its own copy of a clean story — separately, so a location conflict cannot accidentally also be a possession conflict — and the checker is scored against what comes back. Extraction is scored against hand-written passages

with known fact sets. Staleness is scored against a scripted edit whose true blast radius is known because the dependency edges were written by hand, and which deliberately includes a beat that *depends on the changed fact but never declared it* — otherwise the soft-edge ablation could only add false positives, and that would be an unfair test of an idea worth testing properly.

11.3 Results: the deterministic tier

These need no model calls, so they are exact — a regression is a regression, not a bad sampling day — and they cannot be improved by rerunning.

Continuity checker	recall	precision	F1
layer 1 only (deterministic)	80%	100%	0.89
layer 1 + layer 2 (NLI)	100%	100%	1.00

0.00 false positives per beat on a clean story. That number matters more than recall: a checker that flags everything has perfect recall and is worthless, because the writer learns flags are noise and stops reading them. Layer 1 catches location conflicts, deaths followed by action, possession conflicts, and epistemic violations; the one class it cannot decide — two free-text notes that contradict each other — is exactly what layer 2 is for, which is the partition the closed vocabulary was designed to produce.

Staleness prediction	precision	recall	F1
declared edges only	1.00	0.67	0.80
best embedding-edge threshold	1.00	1.00	1.00

Declared edges are exact and blind to what nobody declared. The prediction stated in advance was that embedding edges would trade recall for precision too poorly to be useful; on this case the best threshold reaches 1.00 precision at 1.00 recall (+ soft edges @ 0.68). The case is six beats with lexically distinctive locations, so this is suggestive rather than settled — but it is the opposite of the prediction, and it is recorded as such rather than explained away.

Selector	mean pairwise distance	mean quality
top-k (temperature only)	0.254	0.900
MMR ($\lambda = 0.5$)	0.483	0.833
DPP	0.555	0.780

Six candidates, of which the three highest-scoring are near-paraphrases of each other. The baseline takes all three. This measurement changed the system twice, described in §12.

Preference head, fresh writer	from prior	from uniform	lift
after 5 comparisons	0.770	0.660	+0.110
after 10 comparisons	0.775	0.645	+0.130
after 40 comparisons	0.815	0.835	-0.020

Held-out pairwise accuracy on a persona the prior never saw. The shape is what a population prior should look like: a real advantage while data is scarce, converging to irrelevance as the writer's own comparisons take over.

The Thompson sampler reaches cumulative pseudo-regret 4.8 over 400 rounds against arms at 0.40 and 0.70, and is flat after roughly the first fifty — it pays its exploration cost early and stops. ε-greedy at 0.1 reaches 6.3 and is still climbing linearly, because it explores at a fixed rate forever and explores uniformly. It spent 16 of 400 pulls on the safe arm against 384 on the better one, and its posteriors land at 0.389 and 0.707 against true rates of 0.40 and 0.70.

11.4 Results: the live tier

Four simulated writers drive the real engine through the full configuration. 4 runs, 107 writer decisions, mean acceptance 88%, and 117,336 tokens per decision.

The number that matters is **weight recovery**: the correlation between what the preference head learned and the persona's hidden weight vector. Mean 0.44, best 0.53. Not *did acceptance rise* — a degenerate system achieves that by proposing the same safe thing repeatedly — but *did the machinery recover the taste it was shown*.

Acceptance did not rise, and in three of four runs it fell. The Minimalist held at 100%, the Maximalist went 100% to 91%, the Serialist 91% to 84%, the Controller 100% to 73%. Two runs ended early because their persona rejected all six episode candidates and the harness stops rather than pretending a writer would keep clicking.

That is worth stating plainly rather than framing away, and it is also worth saying what it does and does not show. The comparison is confounded: as a story grows, every new candidate must stay consistent with more established facts, more open threads, and more of the writer's accumulated rules, while the persona's acceptance bar stays exactly where it started. A flat or falling acceptance curve under a rising task difficulty is not evidence that the preference layer failed, and a rising one would not have been evidence that it worked — which is precisely why the headline metric here is weight recovery against a known hidden vector, decided before any of these runs. The first-third figures also sit on seven or eight decisions each, so the endpoints are noisy.

The honest summary of the live tier: the machinery runs end to end against a real provider for a hundred-odd decisions without human intervention, and the preference head recovers roughly half the structure of a hidden weight vector from about twenty-five comparisons. Separating writer satisfaction from rising task difficulty needs a design this evaluation does not have — a fixed held-out decision set replayed at several points in a run, scored by the same persona — and that is the first thing to build if the evaluation gets another pass.

Acceptance and edit-distance curves per persona are in `eval/results/`, beside the full table — including anything that did not run — in `summary.md`. Every number in this section regenerates with

```
python -m eval.run --config full && python -m eval.texnumbers
```

12 What the measurements changed

Two things the design predicted turned out wrong, and both changed the system.

MMR at the conventional $\lambda = 0.7$ was identical to the top-k baseline. Bi-encoder cosine has a high floor — six candidates here span 0.33 to 0.82, and any two English sentences on a related topic score well above zero. A redundancy penalty that is nearly constant across candidates subtracts nearly the same amount from every score, and a constant changes no ordering. At $\lambda = 0.7$ MMR selected at diversity 0.254 and quality 0.900 — exactly the top-k baseline's 0.254 and 0.900. The headline selector was reproducing the ablation it was meant to beat. Fixed twice: the similarity matrix is rescaled within the candidate set, and the default λ moved to 0.5, chosen from a sweep.

The same measurement produced a better argument for DPPs than the one this document originally made. The DPP reaches the diverse answer *with no parameter at all*, because its kernel is multiplicative: a near-duplicate's contribution to the determinant collapses however good it is. MMR's additive penalty can always be outvoted by a large enough quality gap. That is a structural difference rather than a tuning difference.

Embedding-similarity dependency edges did better than predicted, as above.

Both are recorded here rather than quietly folded into the design, because a system that only reports the measurements that agreed with it has not been measured.

13 Decisions, and what was rejected

13.1 Purpose-tag routing rather than per-call-site provider choice

The cost policy is six lines in one table. *Rejected:* Choosing a provider where the call is made. That scatters the policy across forty call sites and makes it impossible to audit or change. It also makes the ablation in §11 — rerunning everything on a stronger model — a one-line edit rather than a refactor.

13.2 Diffs rather than replacement text

Every AI action is a typed operation list against the current state, not a block of prose that overwrites what was there. *Rejected:* Text-level replacement with a visual diff. It looks similar in the interface and is far weaker underneath: a text diff cannot say "this introduces a character", cannot be checked against the world graph before it is applied, and cannot tell the writer what it would invalidate.

13.3 Declared dependencies rather than inferred ones

A beat states what it produces and consumes; propagation walks those edges only. *Rejected:* Inferring dependencies with a model. A dependency oracle that is wrong ten percent of the time makes every propagation result untrustworthy, which is worse than no propagation at all — the writer stops reading the marks. Embedding-similarity edges are implemented, but as a clearly weaker "maybe affected" signal and as an ablation to measure, never as the primary mechanism.

13.4 A closed predicate vocabulary

Ten predicates plus a free-text escape hatch. *Rejected:* Open-vocabulary relation extraction. Closed predicates make the first checker layer a dictionary lookup and an equality test — free, instant, and never wrong for the wrong reason. The escape hatch is exactly the set of facts that needs the more expensive NLI check, which is a useful way to have partitioned the problem.

13.5 Content-addressed snapshots

Rejected: Storing a full copy of the story per version, or an event log without materialized states. Full copies do not survive a few hundred snapshots. A pure event log makes "what is true right now" a replay, and makes branch comparison hard. Content addressing gives cheap versions, free structural sharing, and diff as set arithmetic.

13.6 Axis-conditioned sampling rather than a similarity penalty

Candidates are generated under named instructions and the name reaches the writer. *Rejected:* Sampling k times and reranking for dissimilarity. That is Fabula's approach, and it produces variants that differ without being *nameably* different. The reranker is still there — it stops two candidates that came out similar anyway — but the axes are what make three options a decision rather than three readings.

13.7 A calibrated NLI threshold rather than a chosen one

0.35, measured against the local checkpoint, and swept in the evaluation. *Rejected:* A round number. The interesting case is a real contradiction the model is only moderately confident about (0.43), which a 0.5 threshold silently misses — and silent misses in a continuity checker are the failure that matters.

13.8 Marking rather than auto-repair

Rejected: Automatically regenerating stale downstream nodes. This is the single most tempting feature in the system and the one that would destroy it. It is precisely what Fabula's writers complained about, and the reason is not technical: a system that rewrites approved work without asking cannot be trusted with a manuscript.

13.9 Ten interpretable features rather than a learned representation

Rejected: Embedding the candidate and letting the head learn its own features. With twenty comparisons that head would memorize rather than generalize, and — decisively — its weights would say nothing a writer could read. An interpretable feature vector is what lets the system explain its own ranking back to the person it is ranking for.

13.10 SQLite

Rejected: Postgres. One writer, one story, a few megabytes, and a file the author can copy is the actual workload. Postgres buys concurrency this system does not have. What would change at PocketFM scale is discussed in the systems documentation.

[Further decisions — provider routing, MMR versus DPP, the preference head's feature set, the simulated-writer evaluation — are added as their chunks land.]

14 Limitations

The evaluation cannot measure taste. A persona is a linear functional over ten features plus noise. It cannot be surprised, cannot change its mind, and cannot tell you the tool is exhausting to use. Every number here is a property of the machinery. Whether the system helps a human write a better serial is not established by anything in this document, and only a human study would establish it.

The preference layer is validated against writers built from its own hypothesis class. The head is linear in ten features; the personas score linearly in the same ten. The model is therefore correctly specified by construction, which is why the claim is *recoverability* — the estimator is not broken — rather than *it learns taste*. A real writer's preferences are not linear in ten features.

The prior is synthetic. Nothing in the data it was fitted on ever met a reader. The bet is that the adaptation machinery transfers even though the values do not, and that bet is measured (a +0.13 lift at ten comparisons) rather than assumed.

Injected errors are the errors we thought of. Checker recall is over five known classes. A class nobody imagined is a class nobody measured. The periodic audit exists partly for that, and the honest phrasing throughout is "recall over five classes".

Acceptance over a run confounds two things. Task difficulty rises as a story accumulates facts, threads, and rules, while a persona's bar does not move, so the acceptance curve measures both at once and separates neither. The fix is a fixed held-out decision set replayed at intervals within a run; it is not built, and until it is, the acceptance trend is reported and not interpreted.

Single writer, single story. SQLite, one process, no auth, no concurrency. The scaling discussion is architectural rather than demonstrated.

Development ran on a small free-tier model. Prose quality here is a floor, not a ceiling, and the same evaluation reruns on a stronger model to separate "the system works" from "the model is good".

Fact extraction is a single point of failure. Everything downstream — continuity, propagation, the world-state pane — depends on the world graph being populated correctly. Extraction recall is measured, but a fact that is never extracted is invisible to every layer that follows, and no amount of downstream cleverness recovers it.

References

[1] Prithviraj Ammanabrolu, Wesley Cheung, Dan Tu, William Broniec, and Mark O. Riedl. Bringing stories alive: Generating interactive fiction worlds. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, volume 16, pages 3–9, 2020.

[2] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

[3] Jaime Carbonell and Jade Goldstein. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '98)*, pages 335–336, Melbourne, Australia, 1998. ACM.

[4] Erin Cherry and Celine Latulipe. Quantifying the creativity support of digital tools through the creativity support index. *ACM Transactions on Computer-Human Interaction*, 21(4):21:1–21:25, 2014.

[5] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 4299–4307, 2017.

[6] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *International Conference on Learning Representations (ICLR)*, 2021.

[7] Alex Kulesza and Ben Taskar. Determinantal point processes for machine learning. *Foundations and Trends in Machine Learning*, 5(2–3):123–286, 2012.

[8] Philippe Laban, Tobias Schnabel, Paul N. Bennett, and Marti A. Hearst. SummaC: Re-visiting NLI-based models for inconsistency detection in summarization. *Transactions of the Association for Computational Linguistics*, 10:163–177, 2022.

[9] Piotr Mirowski, Kory W. Mathewson, Jaylen Pittman, and Richard Evans. Co-writing screenplays and theatre scripts with language models: Evaluation by industry professionals. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*, 2023.

[10] Piotr Mirowski, Ben Wedin, Reinald Kim Amplayo, Rich Galt, Duncan Williams, Rida Qadri, Jaume Sanchez-Elias, Erin Drake-Kajioka, Sian Gooding, Lucia Lopez-Rivilla, Joao G. M. Araujo, Lion Schulz, Satinder Baveja, Shakir Mohamed, Edward Grefenstette, Laura Rimell, and Richard Evans. Fabula: Building a narrative storytelling sidekick with the writers' community, 2026.

[11] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pages 27730–27744, 2022.

[12] Daniel J. Russo, Benjamin Van Roy, Abbas Kazerouni, Ian Osband, and Zheng Wen. A tutorial on thompson sampling. *Foundations and Trends in Machine Learning*, 11(1):1–96, 2018.

[13] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding, 2018.

[14] Ann Yuan, Andy Coenen, Emily Reif, and Daphne Ippolito. Wordcraft: Story writing with large language models. In *Proceedings of the 27th International Conference on Intelligent User Interfaces (IUI '22)*, 2022.

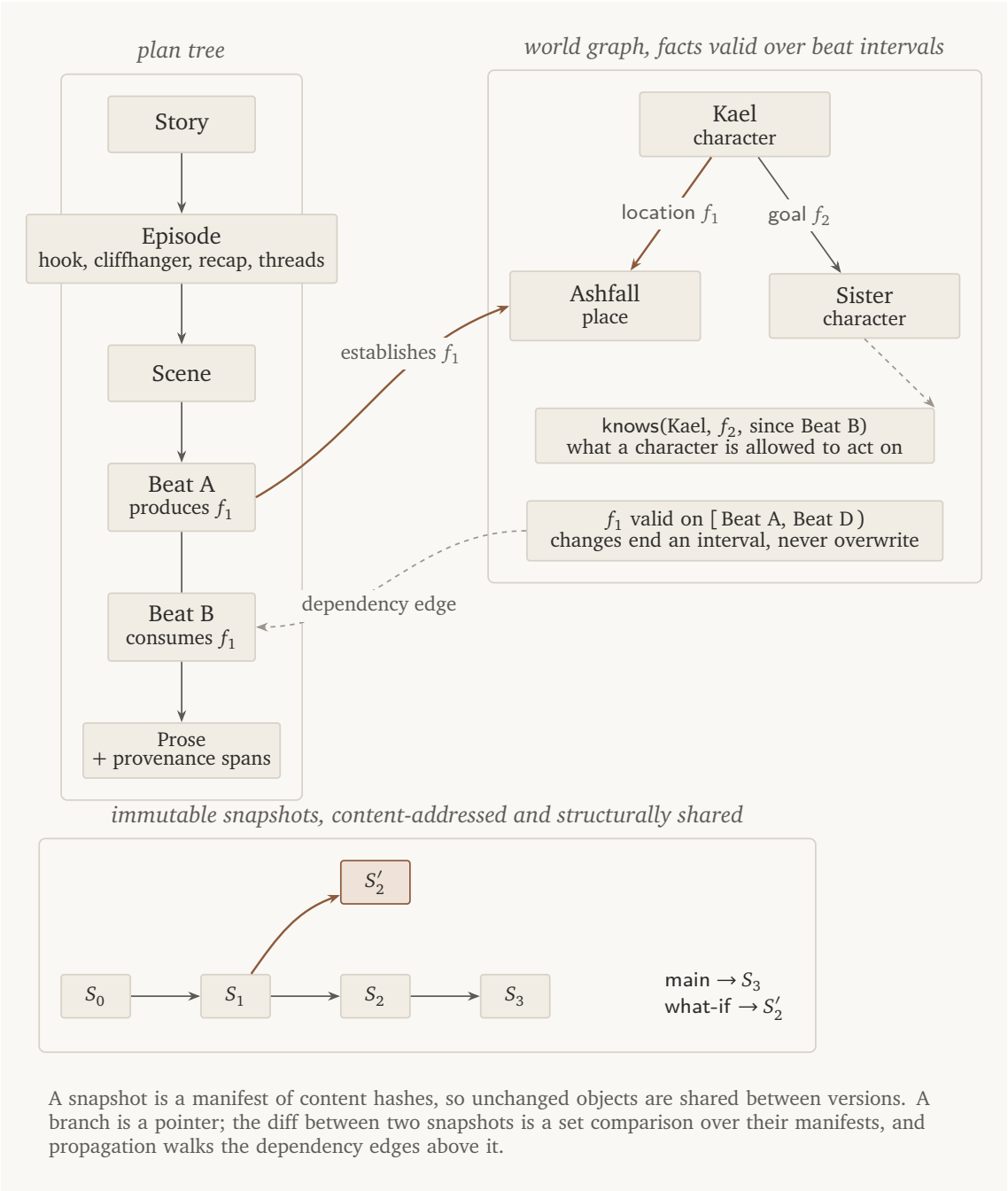

Figure 2: The three stores and how they reference each other. The plan tree holds structure; the world graph holds truth with validity intervals; snapshots hold history. A beat's produces and consumes lists are the only dependency edges propagation walks, which is what makes the walk exact.

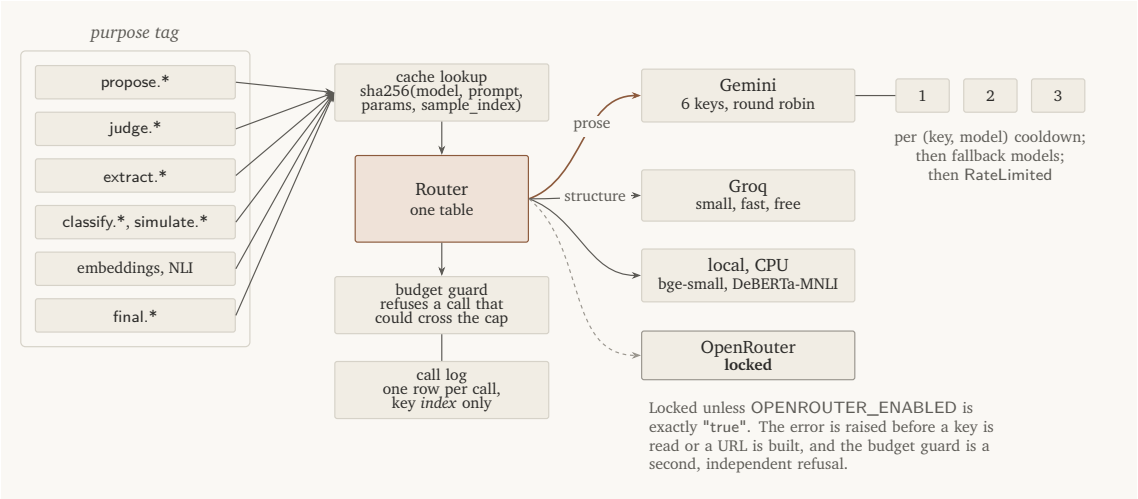

Figure 3: Purpose tags decide the provider; rotation, caching, budget, and logging are layered around it rather than baked into each backend. The metered provider is locked by two independent fail-closed checks.

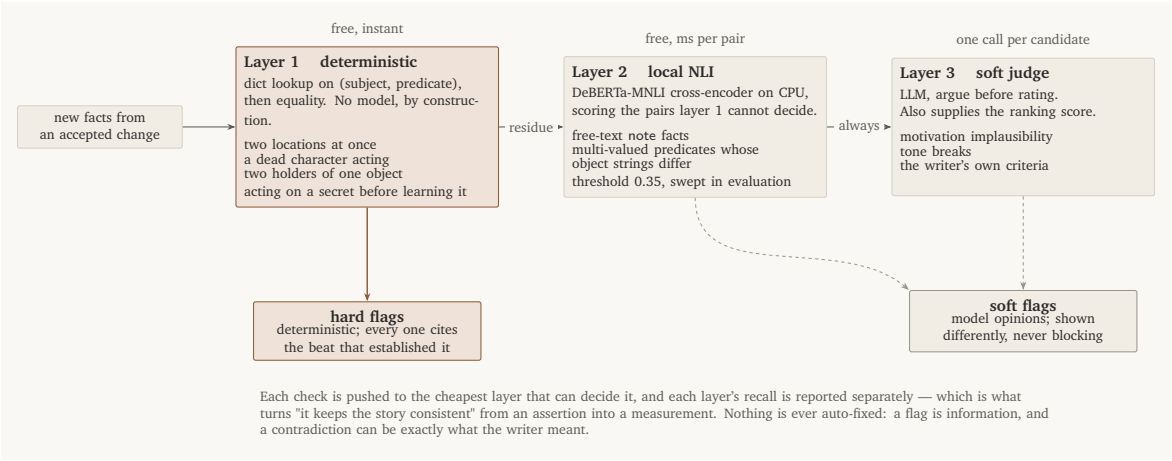

Figure 4: The checker. Layer 1 is deterministic and, by construction, cannot call a model: it imports nothing from the provider layer, and a test parses its imports to prove it. Layer 2 handles only the residue layer 1 cannot decide. Layer 3 is an opinion, labelled as one.

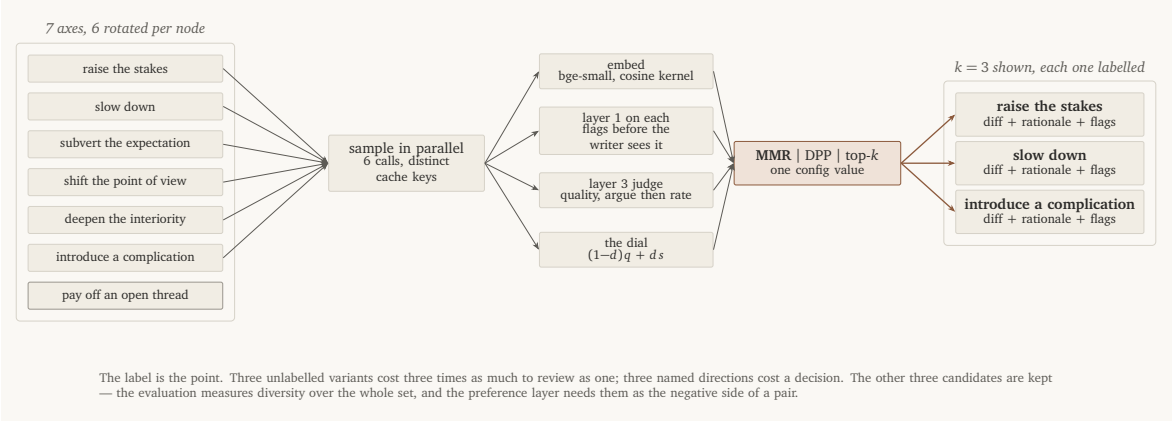

Figure 5: Six axis-conditioned candidates become three labelled ones. The label is the mechanism: three named directions cost a decision, three unlabeled paragraphs cost three readings.

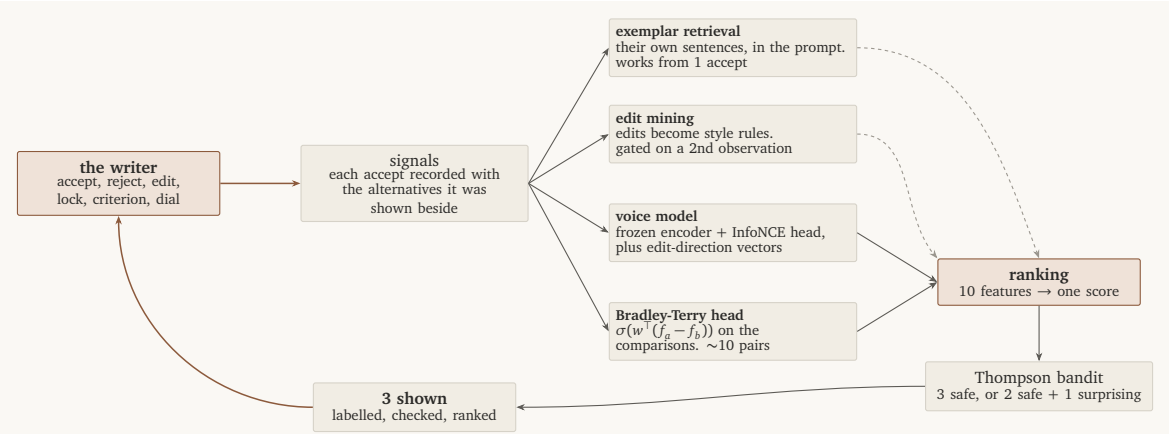

Figure 6: The loop. Every learner degrades to "no opinion" at zero data rather than to a confident guess, which is what makes the no-preference ablation exact. Learned state is persisted per branch, so taste explored on a what-if branch cannot leak back to the main line.

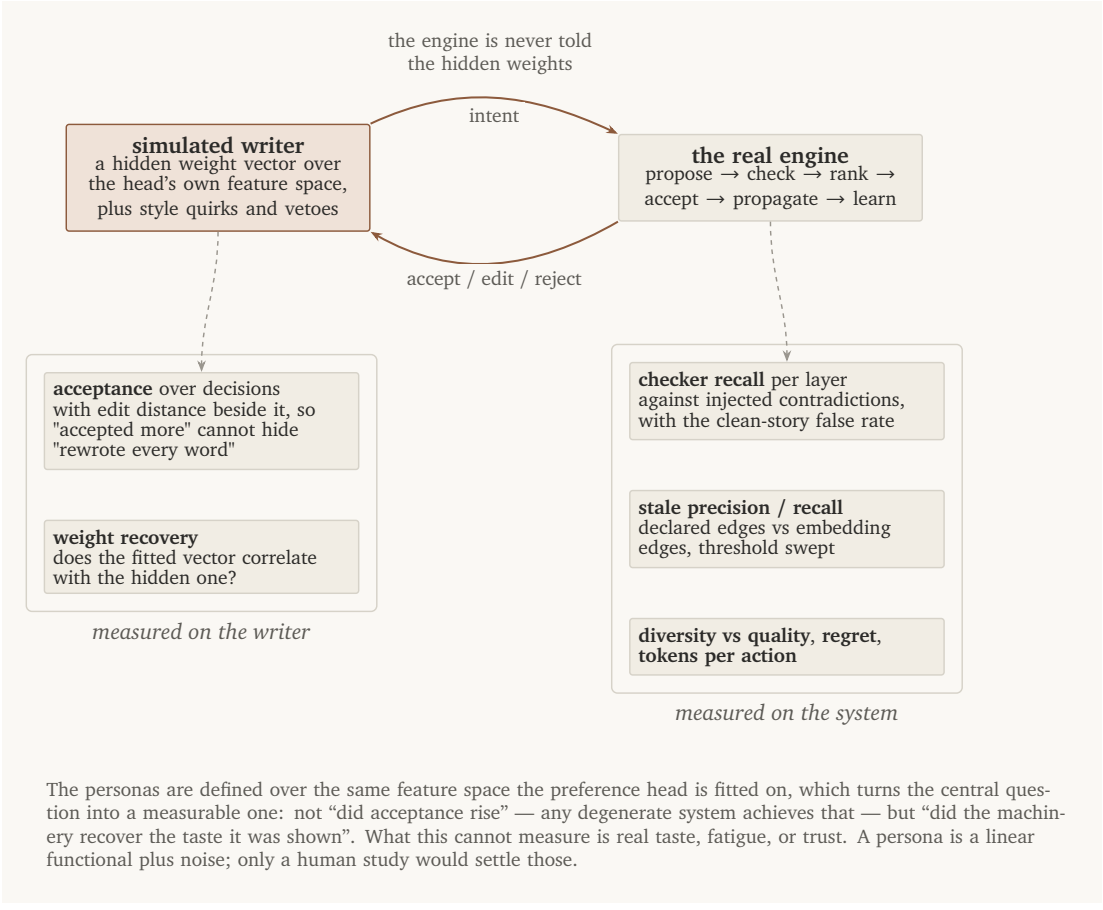

Figure 7: The loop and where the measurements are taken. Metrics on the left are properties of the writer's behaviour; on the right, properties of the system.